

1. Project Name: Sift/Shift

2. Student Names:

Dillon Opperman, Jacob Oswald, Jake Beren, Kristian Neilands, Ben Potter, Josh Schmidt

3. Project Deliverables (Josh Schmidt)

What software/system will be produced? What functionality will it include?

A video game.

Specifically, something similar to Factorio, Shapez, or Beltmatic. The player would oversee a factory which turns materials into other materials/items or changes the form of said materials. At the time of writing our team is unsure as to the exact methods of obtaining the materials and the purpose of refined items. Ultimately, it is a management game where the goal is to create production chains for the factory in an efficient manner.

4. Intended Users (Josh Schmidt)

Who are the users or clients? Are there distinct user roles?

The users would typically be players who enjoy a more casual and strategic gaming experience rather than a fast-paced reactive game. Players who enjoy min-maxing (a term used to describe making the most efficient choices) would be drawn to this game for its management properties. Depending on the milestones we implement, it's possible that players interested in speedrunning will attempt to set records. Our game will hopefully be approachable for players unfamiliar with games like Shapez and provide an experience, again, for casual players.

5. User Problems / Current Alternatives (Josh Schmidt & Jacob Oswald)

What problems do these users currently face? How are they solving them now?

Games like our project can be overwhelming at a glance, and we hope to create something that will appear more approachable. A common issue users can face is the lack of the feeling of progression. It can be troubling to invest money into a game and spend time with it only to feel as though no progress is being made. We aim to create an experience where the player will see their actions matter.

The current alternatives are the games we're using as inspiration for this project; Factorio, Shapez, and Beltmatic. What sets us apart from Factorio is that Factorio is a very deep, very expensive game with very little explanation – it's a great game, but not terribly beginner friendly, especially for people who have never played a factory game before. Shapez and Beltmatic on the other hand are much cheaper and have much less depth but are still paid experiences with upfront costs. With Sift/Shift,

we're aiming for an even more beginner friendly experience with no initial cost barrier, ideally resulting in players getting used to our system and preferring it to other games as they played ours first, and why bother paying a lump sum for a game you already have a version of for free.

6. **System Description / Functional Overview (Ben Potter)**

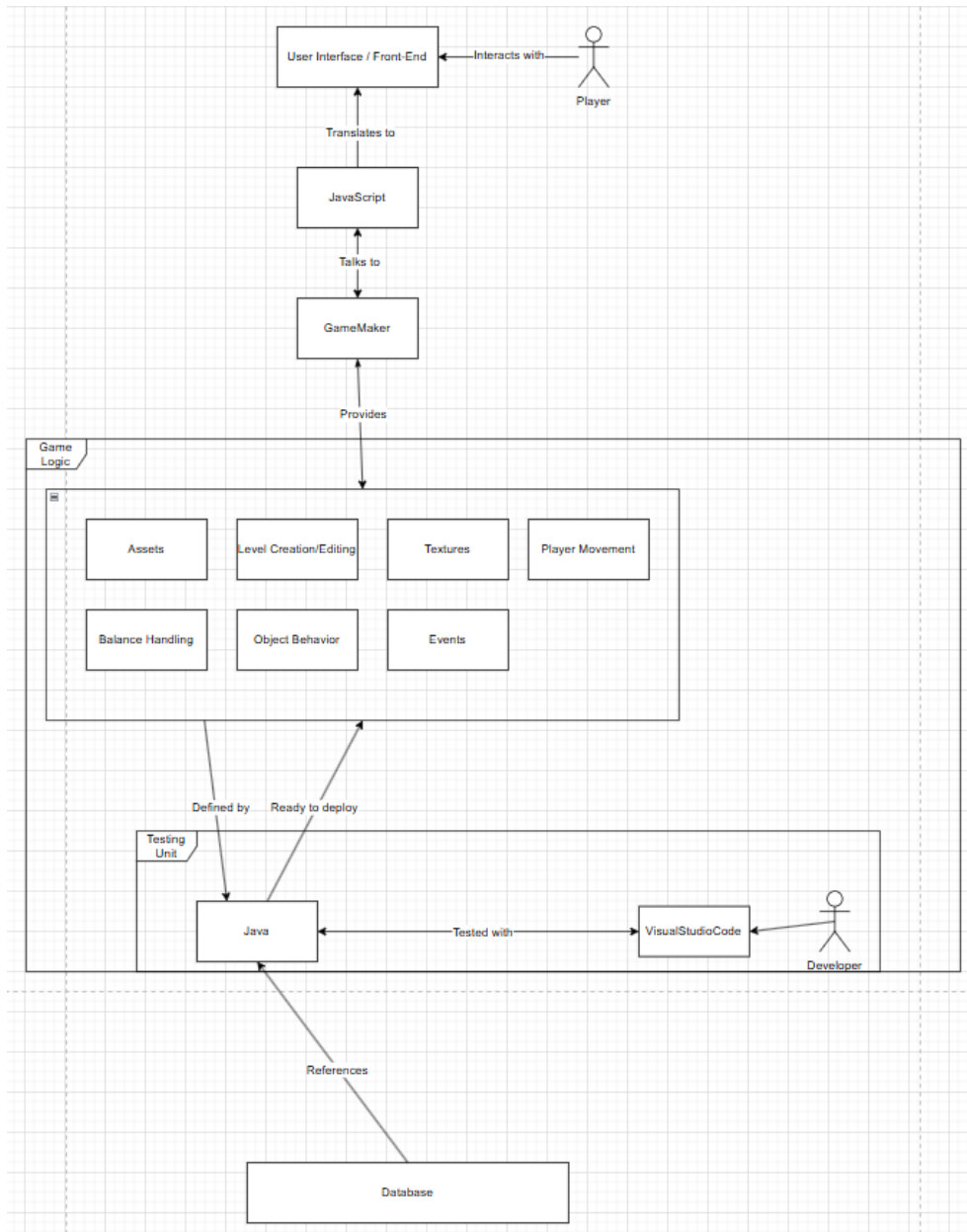
Describe your system in detail — features, workflows, and user interactions.

Some of the features our system will include (some not shown in diagram below):

- Login/Logout
- Account Balance Handling
 - Microtransactions
- Initiatives to play
 - Login bonuses, battle pass
- Level creation/editing
- Time Tracker
- Object behavior
 - Which tiles do what

Our system will consist of a simple request-and-answer type system, wherein the user can request an action (such as moving or placing construction tiles down), and the system will respond to that request.

Generally, this is how the system works on user interaction



It starts by the player user interacting with the UI which then gets passed down the chain of responsibility from the front-end to the back-end. When it arrives to the game engine, the logic of the game is then called based on the context of the player user request. We'll call each function in the game logic "action cases" to simplify this.

The action cases must be defined, which is achieved via Java using the Visual Studio Code IDE where the developer users can test/debug each action case in the testing unit which is also part of the logic portion of the system. Most action cases defined in Java require the use of a database, which will be referenced if necessary.

Once the testing unit phase is complete, the action calls defined in Java are passed back up the chain of responsibility all the way back to the player user where the system finalizes the request.

7. **Build vs. Buy Research (REQUIRED)**

Research existing tools or open-source software. Why not use those? How is yours better or more tailored? Include trade/research citations. (Dillon)

The factory automation and puzzle game market currently features several established titles that share similarities with our proposed system, the most popular ones being Factorio, Shapez and Beltmatic. A more thorough analysis of these existing solutions reveals specific gaps and opportunities that justify custom development rather than licensing or adapting to existing platforms.

Factorio was developed by Wube Software and pretty much represents the standard for factory automation games with 3.5 million copies sold as of 2023 (Wube Software, 2023). The game basically features complex production chains, resource management, and multiplayer capabilities. However, Factorio operates on a premium pricing model with no free-to-play option and maintains a closed-sourced codebase with no licensing opportunities for derivative works (Steam, 2024). The game's complexity also presents a steep learning curve that alienates casual players, with average tutorial completion times exceeding 2 to 3 hours (PCGamesN, 2022). Also, Factorio's engine is proprietary and not available for commercial licensing, making adaptation legally and technically infeasible.

Shapez and Shapez 2 were developed by Tobias Springer to bring a more streamlined approach to factory building with geometric shape processing as the core mechanic (Spring, 2021). While Shapez does provide an open-sourced version on GitHub, the codebase is designed specifically for browser-based game play by using JS and HTML which makes it unsuitable for our java-based development and mobile deployment goals (GitHub, 2024). Furthermore, the monetization model relies on a one-time purchase rather than a free to play microtransaction system central to our project scope. The open-source license would also require our derivative work to remain open source, conflicting with our commercial objectives (Spring, 2021)

Beltmatic, a more recent game, was developed to focus more on minimalist puzzle solving with conveyor belt mechanics, it lacks thorough progression systems, user accounts and online features (Steam 2023). Its offline only architecture makes it fundamentally incompatible with our requirements for user authentication, cloud save functionality, and live service monetization features like login rewards.

After looking at some open-source game engines, similar limitations appear. While engines like Godot (Godot Engine, 2024) and LibGDX (BadLogic Games, 2024) could

theoretically support factory automation mechanics, they provide only foundational frameworks. Implementation of factory specific systems such as conveyor belt physics, item routing algorithms, production chain validation and tile-based construction requires building these mechanics from ground up regardless of engine choice. Studies on game development productivity indicate that genre specific mechanics constitute 60 to 70 percent of development time in simulation games, this basically means that engine selection provides minimal time savings for specialized gameplay systems (Kanode & Haddad, 2009).

The plan for our custom app is to offer several advantages that justify development from the ground up as I mentioned earlier. Our free to play model with “ethical” microtransactions addresses a documented market gap, player surveys indicate 43% of factory game enthusiasts are price sensitive and prefer free entry points (NewZOO, 2023). Also, our streamlined design targets accessibility, aiming for tutorial completion in under 30 minutes compared to Factorio’s multi-hour onboarding. Our architecture integrates live-service features include cloud saves, cross platform progression, time limited events and bonus systems. These features are not in current factory games and have proven to be successful in the gaming industry. (Superdate Research, 2023). Also developing in Java provides cross platform compatibility through JVM deployment.

The technical requirements of our planned system such as user progression and dynamic content updates justify custom backend infrastructure that existing games do not expose or license. Building our own system ensures we maintain full control over the technology stack, data architecture, and monetization implementation while avoiding licensing fees, royalty obligations, and legal restrictions that would accompany adapting existing commercial or open-source solutions.

8. Benefits and Advantages (Jake Beren)

What are the tangible benefits for the users or organization?

Users will be able to play the free demo to understand core gameplay before spending money. This allows a risk free environment not only to get players to discover our game but also to buy the reasonably priced full version/microtransactions. After unlocking the full version users will have permanent access to the full game theoretically allowing for infinite expansion. Due to the creative nature of the game players can feel more connected with their builds long term as well as replaying in a different way.

9. User Requirements & Commitments (Kristian N)

What will the user need to implement your system (hardware, software, training, support)?

Hardware:

Players must have a Windows or MacOS computer that is capable of running a lightweight 2D game. At a minimum, the player must have a dual-core CPU with 8GB of RAM to play comfortably. They must also have a stable internet connection for logins and saves.

Software:

Players must have a current version of any modern internet browser (i.e., Chrome/Edge/Firefox/Safari).

Account & connectivity:

Players must create an account to access things like login bonuses/battle pass progression, saves, and purchased currency/balance tracking.

Training & onboarding:

Player will engage with in-game tutorial, which is designed to be completed fairly quickly (target is within 20-30 minutes). Used for more contextual tooltips; first-time actions like placing, rotating, and deleting items. Also with the use of a 'help' style menu, to show short explanations about tile types and what each does, how item progression.

Support:

Player will have a basic support channel for feedback, or clear escalation path for issues with accounts (specifically purchase/balance issues). Also have a known-issues page or patch notes that will be updated as needed.

Privacy/Data commitments:

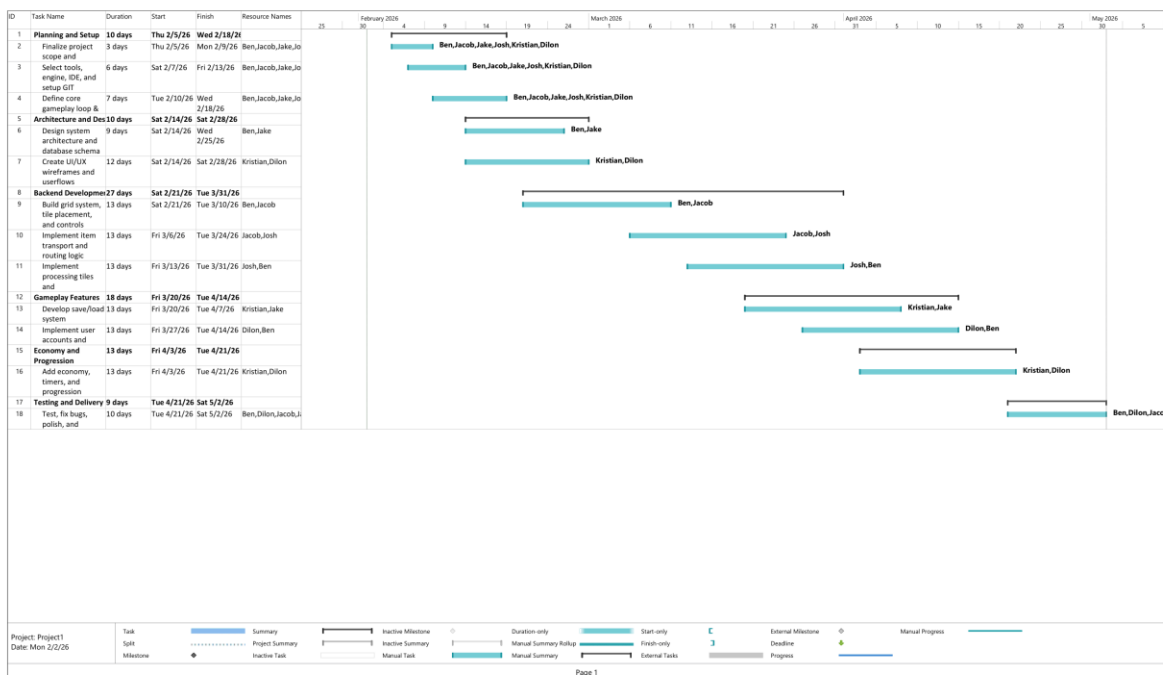
Players will agree that the system stores account credentials securely, game saves and progress, session management, and purchase/balance history.

10. Development Plan (Gantt/WBS-style) (Kristian N)

List 8–15 tasks or milestones with estimated start and end dates. Detail who does what as well (Ben does task A, Josh

Task #	Task Desc.	Approx. Start	Approx. End	Owner
Phase 1: Planning and Setup				
1	Finalize project scope and requirements	Feb 5	Feb 9	Entire Team
2	Select tools, engine, IDE, and setup Git workflow	Feb 7	Feb 13	Entire Team
3	Define core gameplay loop & mechanics	Feb 10	Feb 18	Entire Team
Phase 2: Architecture and Design				

4	Design sys. Architecture and database schema	Feb 14	Feb 25	Ben, Jake
5	Create UI/UX wireframes and user flows	Feb 14	Feb 28	Kristian, Dilon
Phase 3: Backend Development				
6	Build grid system, tile placement, and controls	Feb 21	Mar 10	Ben, Jacob
7	Implement item transport and routing logic	Mar 6	Mar 24	Jacob, Josh
8	Implement processing tiles and transformation logic	Mar 13	Mar 31	Josh, Ben
Phase 4: Gameplay Features				
9	Develop save/load system	Mar 20	Apr 7	Kristian, Jake
10	Implement user accounts and saves	Mar 27	Apr 14	Dilon, Ben
Phase 5: Economy and Progression				
11	Add economy, timers, and progression systems	Apr 3	Apr 21	Kristian, Dillon
Phase 6: Testing and Delivery				
12	Test, fix bugs, polish, and prepare final demo	Apr 21	May 2	Entire Team



11. Team Resource Needs (Jake Beren)

What tools, frameworks, or resources will you need — and do you have access?

Tools: The primary development language we will be using is Java, which can be conducted using Visual Studio Code, which is free and everyone has access to. For version control and collaboration, we will use a GitHub private repository which is also free and accessible by everyone.

Engine: Building our own custom logic layer in Java allows for more control over gameplay mechanics and monetization system. This system with the possible use of Java-based libraries would be free but requires more time spent on building the engine. On the other hand using a prebuilt engine like GameMaker would allow for less time spent on the engine and more on the finer details of the game, though monetization would not be possible.

Backend: We will use a relational database most likely MySQL which the free version should be enough to keep track of player accounts, cloud saves, progression tracking, and microtransactions handling.

References

- Blog*. (n.d.). Factorio. Retrieved February 1, 2026, from <https://factorio.com/blog/>
- Engine, G. (n.d.). *Godot Engine—Free and open source 2D and 3D game engine*. Godot Engine. Retrieved February 1, 2026, from <https://godotengine.org/>
- Kanode, C. M., & Haddad, H. M. (2009). Software Engineering Challenges in Game Development. *2009 Sixth International Conference on Information Technology: New Generations*, 260–265. <https://doi.org/10.1109/ITNG.2009.74>
- libGDX*. (2025, December 14). libGDX. <https://libgdx.com/>
- PCGamesN | PC games news, reviews, guides, and hardware*. (n.d.). Retrieved February 1, 2026, from <https://www.pcgamesn.com/>
- tobspr Games. (2026). *Tobspr-games/shapez.io* [JavaScript]. <https://github.com/tobspr-games/shapez.io> (Original work published 2020)
- tobspr.io, tobspr G.-. (n.d.). *Shapez*. Retrieved February 1, 2026, from <https://shapez.io/>
- NewZoo. (2023). *Global games market report: Player insights and monetization trends*. NewZoo Market Analytics.

Steam. (2023). *Beltmatic* [Video game]. KittenPetter Studios.

Steam. (2024). *Factorio* [Video game]. Wube Software.

Superdata Research. (2023). *Worldwide digital games market: Year in review*. Nielsen Holdings.